University of Glasgow | School of Computing Science

Friday 26 April 2019
09.30 am – 11.00 am

(Duration: 1 hour 30 minutes)

EXAMINATION FOR

DEGREES OF MSci, MEng, BEng, BSc, MA and MA (Social Sciences)

Big Data (H)

Exam Rubric: Answer All Questions

This examination paper is worth a total of 60 marks

You must not leave the examination room within the first hour or the last half-hour of the examination.

INSTRUCTIONS TO INVIGILATORS

Please collect all exam question papers and return to School together with exam answer scripts

Question 1 — 18 Marks

A. GFS/HDFS uses blocks of size 128MB by default. What is the main reason for choosing such a large block size, given the use cases these systems were designed for? [3 points] Briefly explain your answer. [5 points]

a. To minimise disk seek overhead
b. To minimise network bandwidth usage
c. To increase read/write parallelism
d. To reduce the storage load on DataNodes

[8 points]

[Bookwork + Synthesis]
Answer: (a) [3 points]
The disk seek time is fixed and significant (~10ms in standard rotational disks, when the typical transfer rate is 100MB/s). The more data we read per seek (i.e., the larger the block size), the less the disk seek cost/overhead becomes. At 128MB, the seek time is roughly only 1% of the transfer time. [2 points]
Other than that, the block size has no effect on network bandwidth usage (as we routinely scan all data, the same amount of bytes will be accessed over the network), read/write parallelism (assuming datasets much larger than the block size, the parallelism bottleneck is typically in the number of worker nodes) or storage load on DNs (the disk usage is the same, regardless of into how many blocks the data is partitioned). [3 points, one per incorrect answer]
[Full marks will also be awarded if the latter (reduction ad absurdum) part is adequately developed]

B. The default replica placement policy for HDFS places the first copy of a block on the local DataNode (DN1), the second copy on a DN2 located on the same rack as DN1, the third copy on a DN3 on a rack other than the one where DN1 and DN2 are located, and all subsequent copies on random DNs around the cluster. This design strikes a trade-off between/among which of the following? [3 points] Briefly explain your answer. [7 points]

a. Disk/memory storage load on the NameNode, reliability, and network congestion
b. Reliability and read/write network bandwidth consumption
c. Remote read/write latency, NameNode storage load, and DataNode processing load
d. Ease of choosing replication targets and DataNode storage load balancing

[10 points]

[Bookwork + Synthesis]
Answer: (b) [3 points]
Transfers for nodes on the same rack go through only two network hops (node A to top-of-rack switch to node B), while transfers across racks require much more network hops (depending on actual cluster network topology). More hops mean higher latency but also higher bandwidth usage across the cluster. Having the first copy on the local node means zero write bandwidth overhead. Having the second replica on DN2 means there is no single-point-of-failure (SPoF) in DN1, while the write bandwidth usage to create a 2nd copy is minimal (limited to the rack). Having DN3 on another rack means there is no SPoF in the rack of DN1/DN2, for the cost of a single transfer of the block. Multiple copies around the cluster also mean that the read bandwidth consumption has a higher potential of being minimised. [7 points]
Alternative explanation: Having all replicas on one node would minimise write bandwidth consumption, but there would be no real reliability (single point of failure), while the average off-rack read bandwidth cost would also be high (all off-rack nodes requiring multiple hops to the data). Having each replica on a different DN/rack would guarantee maximum reliability and potentially minimal read network bandwidth consumption, at the cost of much higher write network bandwidth consumption. Last, having the first two replicas on the same rack keeps the overall replication network consumption down and reduces the chances for needing all off-rack replicas. [7 points]

[Full marks will also be awarded for adequately developed reduction ad absurdum answers. Partial marks for covering part of the discussion above.]

Question 2 — 21 Marks

A. Assuming only one job executes at your cluster at any given time, what is the optimal number of reducers (with regards to minimising the overall job time) to be used for a WordCount-type job over a large dataset? Assume that the number of unique words is several orders of magnitude higher than the number of nodes in your cluster. [3 points] Briefly explain your answer. [5 points]

a. 1
b. As many as there are keys in the final output
c. As many as there are computers in the cluster
d. A small multiple of the number of CPU cores across all cluster nodes

[8 points]

[Problem solving/Synthesis]
Answer: (d) [3 points]
To optimise this job we need to make as full a use of the cluster's resources as possible. Thus (a) is obviously wrong. Option (b) would lead to too many reducers, each being assigned a single key; this would result in high CPU overhead for instantiating the reducers, hence (b) is suboptimal. Option (c) achieves better parallelism than (a) but doesn't take into account the amount of processing resources (CPU cores) of the nodes, leading to underutilising the cluster CPU capacity. Option (d) is thus the correct option; it achieves maximum parallelism, and the small multiple factor allows for load imbalances across reducers to be evened out across the reduce tasks. [5 points]
[Full marks will also be awarded for complete reduction ad absurdum answers as above, or for answers adequately explaining the last point. Partial marks for covering part of the discussion above.]

B. Which is the main disadvantage of performing a reduce-side join? [3 points] Briefly discuss your answer. [3 points]

a. Inability to support joining more than two datasets/tables
b. Inability to support composite keys
c. Larger I/O costs because of shuffle and sort
d. The results may occasionally be wrong, depending on the cardinalities of the joined datasets/tables

[6 points]

[Problem solving/Synthesis]
Answer: (c) [3 points]
Reduce-side joins require that the input datasets are rehashed (i.e., completely shuffled around the cluster) on their join key, leading to a very high network and disk I/O cost for the shuffle phase. Other than that, reduce-side joins have the same semantics/capabilities as all other types of joins supported by MapReduce, hence options (a), (b) and (d) are incorrect.
[Full marks will also be awarded if either of the parts of the above discussion are present (e.g., discussion of reduce-side join cost, or reduction ad absurdum. Partial marks for covering part of the discussion above.]

C. Given the following Spark program, identify at which step (i.e., line of code) the input RDD is actually computed. [2 marks] Briefly explain your answer. [2 marks]

```
1 JavaRDD<String> inputRDD = sc.textFile("sample.csv");
2 JavaRDD<String> fooRDD = inputRDD.filter(line=>line.contains("FOO"));
3 JavaRDD<String> colsRDD = fooRDD.map(line=>line.split(","));
```

```
4 int count = fooRDD.count();
```

[4 points]

[Bookwork/Problem solving]
Answer: (4) [2 points]
The first three lines only contain transformations. RDDs are only computed for actions, and count() is the only action in the above code. [2 points]

D. You are given an input RDD that contains the edges of a large graph, with each edge being a pair: <source node id> <destination node id>. You are asked to design a job that will produce an output containing pairs in the format: <node X> <list of nodes that have out-edges leading to node X>. Which of the following will produce the correct result?

a. input.groupByKey()
b. input.countByKey()
c. input.map(x => (x._2, x._1)).groupByKey()
d. input.map(x => (x._2, x._1)).countByKey()

[3 points]

[Bookwork/Problem solving]
Answer: (c) [3 points]

Question 3 — 21 Marks

A. Consider the CAP Theorem. What kind of system would you choose to implement a high-volume online chat application and what to store small but highly important (e.g., banking/financial) data? [3 points] Briefly explain your answer. [5 points]

a. Chat: CA, Banking: AP
b. Chat: CA, Banking: CA
c. Chat: CP, Banking: AP
d. Chat: CP, Banking: CA

[8 points]

[Bookwork/Problem solving]
Answer: (d) [3 points]
A high-volume chat application would sacrifice availability (it's ok if messages are occasionally delayed), for consistency (all chat participants have the same view) and partition tolerance (its volume would require a large-scale deployment, hence partitions would be the norm). The banking application, on the other hand, would require high consistency and availability; as the data is said to be small, a centralised setup would be preferable, hence partition tolerance isn't necessary. If a distributed deployment is necessary, the application should sacrifice availability in the face of network partitions to guarantee consistency. [5 points]
[Full marks will also be awarded for complete reduction ad absurdum answers. Partial marks for covering part of the discussion above.]

B. In systems following the BigTable design, when a tablet/region grows bigger in size than the predefined maximum region size, it:

a. Gets compressed using a predefined compression codec
b. Spills into adjacent machines
c. Is discarded and an appropriate error is returned to the user
d. Is split into smaller regions

[3 points]

[Bookwork]
Answer: (d) [3 points]

C. The tuple which uniquely identifies a cell in on-disk BigTable/HBase storage is:

a. {rowkey, column qualifier, timestamp}
b. {rowkey, column family, timestamp}
c. {table, rowkey, column family, column qualifier}
d. {column family, column qualifier, timestamp}

[3 points]

[Bookwork]
Answer: (a) [3 points]

D. Assuming no cluster node faults and a client external to the cluster, which of the following Cassandra consistency levels would accomplish read-your-own-write consistency (i.e., a client sends an update for a key and immediately afterwards a read for the same key, and the result should be the same as the one written in the first operation) with minimum write latency for a replication factor of 5? [3 points] Briefly explain your answer. [4 points]

a. Write: ANY / Read: ONE
b. Write: ONE / Read: ONE
c. Write: QUORUM / Read: QUORUM
d. Write: ALL / Read: ONE

[7 points]

[Bookwork/Synthesis]
Answer: (c) [3 points]
As the client is external to the cluster, it will need to contact a cluster node that will the act as a proxy. The first two answers cannot guarantee read-my-own-write semantics as only one copy is placed on the cluster and a different replica could be selected for the read. Both options (c) and (d) can guarantee the required semantics, but the latter has much higher write latency. [4 points]
[Partial marks for covering part of the discussion above.]